

Pomysły WMS — backlog

Brain-dump pomysłów posegregowany wg charakteru pracy. Zaktualizowano: 2026-07-21.

Legenda: [] do zrobienia · [x] zrobione · ● szybka wygrana · ● średnie · ● większe / decyzja projektowa.

1. Funkcje aplikacji (nowe / rozbudowa)

- ● **Kafle palet na pulpicie** — na Pulpicie (Faza 5, `routes/pulpit.js` + `public/desktop`) kafle typu „wolne palety” klikalne → filtrowana lista lokalizacji (`typ=paleta`, wolne/stan 0). Wymaga nowego agregatu w `/api/pulpit` i widoku listy. Typ lokalizacji liczy `services/lokalizacje-model.js` (K4G → zawsze paleta).
- ● **Waga gabarytowa z kartonów + edytowalna lista kartonów w adminie** — masz `config/kartony.js` (32 kartony, `dobierzKarton`, `gab` = vol/4000), ale feature jest niepodpięty. Dwa kroki: **(a)** podpiąć dobór kartonu pod liczenie wagi gab. (dziś liczy ją `services/gt-atrybuty.js` z wymiarów produktu ÷4000); **(b)** przenieść listę kartonów z pliku `config/kartony.js` do edycji w panelu admina (nowy route + widok). Patrz `MEMORY.md` → „Kartony: config i analiza”.
- ● **Auto-czyszczenie zer po 30 dniach bez ruchu** — ▲ **zmiana decyzji projektowej, nie zwykły task**. Ścieżka 3 „Czyść zera” świadomie NIE ma warunku „X dni bez ruchu” (decyzja 2026-07-19), a inwariant „Lokalizacja K4 przeżywa stan 0” mówi, że dom K4 wolno skasować **tylko człowiekowi przy regale** (`POST /api/sciezki/czysc-zera/zwolnienie` — jedyne dozwolone miejsce). Automat po czasie łamie to założenie: wnioskuje ze STANU, a zero znaczy „półka pusta”, nie „towaru już nie ma”. **Do świadomej decyzji: automat czy dalej ręcznie**. Historia w bazie zostaje (`audyt`), więc argument usera jest sensowny — ale to trzeba rozstrzygnąć, nie wpisać po cichu. Szczegóły zasady: `docs/zasady.md`.
- ● **Przebudowa odkładania zwrotów — najpierw lokalizacja, tożsamość widoczna** — ZROBIONE 2026-07-22 (`public/zebra/zwroty.js` + `ruch.html` + `app.css`, bez zmian backendu). Powód (obchód z magazynierami): stary „skan towaru → marsz → skan lokalizacji” gubił tożsamość po marszu („nie wiem który towar”). Odwrócony przepływ na Zebrze: (1) ekran prowadzący — DUŻA lokalizacja docelowa + towar (SKU podpis nad dużą nazwą) + wyeksponowana ilość + rezerwacje/zestawy; (2) skan **lokalizacji** = „jestem na miejscu”; (3) skan **produktu** (lub „nie skanuje się”) = „to ten”; (4) ilość + Odlóż. Nazwa i lokalizacja widoczne cały czas — nic się nie zwiija. Nagłówek „Wózek ...” zdjęty z ekranu pozycji (numer w linii postępu) → całość mieści się w 360×536 bez scrolla. Zweryfikowane w izolowanym podglądzie obu ekranów; **do przetestowania na realnej Zebrze**. Opcja później: miniatura zdjęcia produktu (dane z GT).
- **Ochrona domu WMS przy zmianie lokalizacji (2026-07-22)**. Gdy skan ≠ dom WMS (`zwroty.js` `decyzjaLokalizacji`, kontekst z `/k4-dom` + `/skan`): **K4>0 lub zapas(K4+K4G+LS)>0** → **blok** („odłóż na dom X”, bo dom żywy / towar wróci z K4G/LS); **zapas=0** → **confirm** „Jesteś na złej lokalizacji (X, stan 0) — zmienić?” i **atomowe przeniesienie domu** (backend `POST /ruchy/rozloz` {`przenies_dom:true`} kładzie na nowej i zwalnia stary pusty wpis K4 — drugie po „Czyść zera” miejsce kasujące dom, autoryzowane człowiekiem przy regale); **brak domu WMS (tylko GT)** → **info** „ustalasz miejsce 1. raz”. Pliki: `zwroty.js` + `routes/ruchy.js`. 5 przypadków zweryfikowanych w podglądzie; backend do sprawdzenia na realnej bazie.
- **Zachowanie zapasu GT□only przy rozkładaniu (2026-07-22)**. `synchronizujLokalizacje` (`services/gt-fields.js`) miał strażnik „nie nadpisuj `tw_Pole8` (K4G) dopóki cały stan GT nie rozłożony w WMS”, ale `tw_Pole1` (K4) **zawsze nadpisywał** — przy rozkładaniu SKU z lokalizacją tylko w GT gubił zapas trzymany w GT (`A1/P5` → `A1`, bo `P5` nie ma odpowiednika w WMS `zapas_kod`). Dołożony **symetryczny strażnik K4**: `tw_Pole1` nietknięty, dopóki `GT K4 - suma WMS > 0` (deficyt) — pole i zapas w GT przeżywają do pełnego rozłożenia. Pokrywa przypadek częściowego

rozłożenia (deficyt>0). 54/54 testy zielone; do potwierdzenia na realnym „nerchiok”.

2. Drobne UX na istniejących ekranach

- ● Ścieżka „Brak parametrów” → skok do wagi gdy są wymiary — ZROBIONE 2026-07-21
(`public/zebra/parametry.js, otworz()`). Gdy wszystkie trzy wymiary są komplet (>0), focus ląduje od razu na polu wagi; przy braku wymiarów (lub złych danych typu `0×65×53`) zaczyna normalnie od długości.
- ● Desktop: pokaż lokalizację K4G w edycji — w edycji produktu na desktopie
(`public/desktop/app.js`) w trakcie wpisywania znika info, gdzie towar leży na K4G, i trzeba się cofać. Pokazać lokalizację K4G obok pola edycji (K4G = 1 SKU = N lokalizacji, więc jest co pokazać — źródło: `stany_lokalizacji / tw_Pole8`).

3. Infrastruktura i bezpieczeństwo danych

- ● Backup lokalizacji + bazy do chmury — `db/wms.db` + eksport lokalizacji na wypadek padu peceta.
Jest już `services/backup.js` (lokalny job, `WMS_BACKUP_DISABLED` w `launch.json`) — do rozbudowy o kopię poza maszynę (chmura/zdalny dysk). WMS = jedyne źródło lokalizacji, więc ma najwyższy priorytet z tej trójki. Patrz `MEMORY.md` → „Produkcja: deploy na pececie”.

4. Dev / dokumentacja / jakość (nie sama apka)

- Rozrys architektury aplikacji — ZROBIONE 2026-07-21 → `docs/architektura.md` (diagramy mermaid: kontekst systemu, dwa kanały do GT, przepływ ruchu, joby).
- Przegląd zasad per element (dokumentacja) — ZROBIONE 2026-07-21 → `docs/zasady.md` (zasady nadrzędne, inwarianty + gdzie egzekwowane, magazyny, pola własne, ścieżki, parametry, joby).
- Code review na najlepszym modelu — `/code-review ultra` (multi-agent w chmurze, całej gałęzi). Uruchamiasz Ty — jest płatne, nie odpalę go sam. Alternatywa lokalna: `/code-review high`.

Od czego zacząć

1. ● Desktop K4G w edycji — drobny UX, zero ryzyka, realnie irytuje przy codziennej pracy.
2. ● Waga z kartonów (a) — masz połowę roboty w `config/kartony.js`; podpięcie doboru kartonu to niewielka zmiana w `gt-atrybuty.js`.
3. ● Decyzja o auto-czyszczeniu zer — zanim cokolwiek kodować, rozstrzygnij tension z inwariantem (patrz punkt wyżej).

Backup (#9) i edytowalna lista kartonów (#4b) to większe kawałki — warto zaplanować osobno.